Поточные шифры — это симметричные криптосистемы, которые шифруют открытый текст посимвольно (обычно побитово или побайтово). В основе лежит генерация псевдослучайной последовательности — **гаммы** (keystream), которая затем накладывается на открытый текст с помощью обратимой операции (как правило, сложения по модулю 2:

$$C_i = P_i \oplus \gamma_i$$

Чтобы поточный шифр был надежным, быстрым и практичным, при его проектировании соблюдается ряд криптографических и инженерных принципов.

---

1. Период гаммы должен быть достаточно большим

Генератор гаммы является конечным автоматом, а значит, рано или поздно его выходная последовательность начнет повторяться. Период T этой последовательности должен быть достаточно большим (например, $T \geq 2^{64}$ или 2^{128}), чтобы скючить возможное повторное использование. Повышает стойкость

- **Для чего нужен:**
  - Повышает стойкость т.к. реализуется защита от **«проблемы повторного использования блокнота» (Two-Time Pad)**. Если один и тот же участок гаммы шифрует два разных блока текста ($C_1 = P_1 \oplus \gamma$ и $C_2 = P_2 \oplus \gamma$), то $C_1 \oplus C_2 = P_1 \oplus P_2$. Это полностью снимает криптографическую защиту и позволяет восстановить тексты методом частотного анализа.

2. Принцип высокой линейной сложности (Linear Complexity)

Повышает стойкость. Линейная сложность последовательности — это длина самого короткого линейного регистра сдвига с обратной связью (LFSR), способного сгенерировать эту последовательность. Она должна составлять не менее половины длины рассматриваемого отрезка гаммы. Также повышает стойкость.

- **Для чего нужен:**
  - Защита от **алгоритма Берлекэмпа — Мэсси**. Если линейная сложность равна L, то криптоаналитик, перехватив всего $2L$ бит гаммы, может однозначно восстановить структуру генератора и вычислить всю последующую (и

предыдущую) гамму. Высокая линейная сложность лишает злоумышленника этой возможности.

3. Принцип статистической псевдослучайности (Критерии Голомба)

Повышает стойкость. Выходная гамма должна быть статистически неотличима от абсолютно случайной последовательности (т.е. от “белого шума”). На практике это означает:

- Равное количество 0 и 1 (сбалансированность).
- Равномерное распределение блоков («серий») одинаковых бит.
- Нулевую автокорреляцию (сдвиг последовательности относительно себя не выявляет закономерностей).
- **Для чего нужен:**
  - Защита от **статистического криптоанализа**. Если в гамме преобладают единицы или есть шаблоны (как это было в ранних версиях RC4), злоумышленник может извлечь статистическую информацию об открытом тексте из шифротекста без знания ключа.

4. Принцип нелинейности и корреляционной стойкости (Non-linearity & Correlation Immunity)

повышает стойкость. В схему генератора обязательно вводятся сложные нелинейные элементы (нелинейные регистры NLFSR, нелинейные функции фильтрации, S-блоки, ARX-операции — сложение/вращение/XOR). Функция выхода должна иметь высокий порядок корреляционной стойкости (теорема Зигенталера).

- **Для чего нужен:**
  - **Против методов линейной алгебры:** Чисто линейные системы (например, обычный LFSR) тривиально взламываются решением системы линейных уравнений.
  - **Против корреляционных атак (метод «Разделяй и властвуй»):** Если выходной бит хотя бы слабо коррелирует с состоянием одного из внутренних регистров, криптоаналитик может вскрывать систему по частям, а не перебирать всё пространство ключей целиком.

5. Принцип достаточного размера внутреннего состояния (Resistance to TMDTO)

Повышает стойкость. Размер внутреннего состояния генератора (S) должен быть **как минимум в два раза больше длины ключа (K)**: $|S| \geq 2|K|$.

- **Для чего нужен:**
  Если размер состояния равен длине ключа ($|S| = |K|$), атакующий может восстановить состояние за время $O(2^{|K|/2})$ вместо полного перебора $O(2^{|K|})$. При $|S| \geq 2|K|$ сложность атаки поднимается до гарантированного уровня стойкости ключа.

6. Принцип стойкости к алгебраическим атакам (Algebraic Immunity)

Повышает стойкость. Уравнения, связывающие биты ключа/состояния и биты гаммы, должны иметь высокую степень нелинейности и не должны редуцироваться (упрощаться) до систем уравнений низких степеней.

- **Для чего нужен:**
  - Защита от **алгебраических атак (методы базисов Грёбнера, алгоритмы XL/XSL)**. Если криптосистему можно описать системой уравнений низкой степени, современные математические солверы могут найти ключ за полиномиальное или сублогарифмическое время.

7. Принцип безопасной инициализации (Key and IV Setup / Diffusion)

Повышает стойкость, повышает эффективность. Фаза инициализации (смешивание секретного ключа K и вектора инициализации IV) должна обладать мощным **лавинным эффектом** (т.н. “Avalanche effect”). Для этого генератор делает несколько десятков «пустых» тактов (warm-up rounds) без выдачи гаммы.

- **Для чего нужен:**
  - Позволяет использовать один и тот же ключ многократно с разными IV.
  - Защита от **атак по связанным ключам (Related-Key Attacks)** и **атак по схожим векторам инициализации (Slide Attacks)**. Изменение даже 1 бита в ключе или IV должно приводить к абсолютно непредсказуемому изменению всего внутреннего состояния до выдачи первого бита гаммы.

8. Принцип аппаратной и программной эффективности

Повышает эффективность. Алгоритмы должны оптимально использовать доступные вычислительные ресурсы:

- *В «железе» (Hardware-oriented, например, Grain, Trivium):* минимальное количество логических вентилей (GE — Gate Equivalents), низкое энергопотребление.
- *В «софте» (Software-oriented, например, ChaCha20, SNOW 3G):* использование 32- и 64-битных регистров CPU, параллелизма (SIMD/AVX), операций сложения, сдвигов и XOR (ARX).
- **Для чего нужен:**
  - Поточные шифры часто применяются там, где блочные шифры слишком «тяжелы»: в IoT-устройствах, смарт-картах, RFID-метках, беспроводных сетях реального времени (GSM/5G) и при передаче потокового видео, где критически важны **минимальная задержка (latency)** и **высокая скорость**. Чем ближе к “железу” - тем лучше.

9. Принцип синхронизации и устойчивости к ошибкам (Выбор модели)

Повышает эффективность. Архитектурно шифры делятся на:

1. *Синхронные:* гамма генерируется независимо от шифротекста.
2. *Самосинхронизирующиеся (асинхронные):* гамма зависит от N предыдущих бит шифротекста.

- **Для чего нужен:**
  - **В синхронных шифрах:** Ошибка при передаче 1 бита шифротекста портит ровно 1 бит открытого текста (нет «размножения ошибок»), что критично для каналов связи с помехами (голосовая связь).
  - **В самосинхронизирующихся шифрах:** При потере/вставке бита (рассинхронизации) система автоматически восстанавливает расшифрование через N шагов, не требуя повторного протокола рукопожатия (handshake).

Некоторые выводы

Идеальный поточный шифр — это компромисс: он использует **математические свойства** (период, нелинейность, алгебраическую стойкость) для обеспечения **абсолютной вычислительной стойкости**, сохраняя при этом **минимальную алгоритмическую сложность** для обеспечения сверхвысокой скорости работы.
Также надо понимать, что потоковые шифры на практике используется не сами по себе а на основе тех или иных алгоритмов, “зашитых” в crypto-api ОС.

Поддержка работы таких алгоритмов сильно зависит от требований безопасности и поддержки их на уровне криптоапи. Например, шифр RC4 вполне может и не поддерживаться рядом браузеров.

Стоит отметить, что поточные системы шифрования могут быть весьма эффективные, если их используют злоумышленники для шифрования файлов т.к. им нет нужды соответствовать всем указанным требованиям - достаточно реализовать некий “базовый” функционал шифра.